

AMU | Language, Mind, & Tech
Programming in Python
Ali Asgari



Ant/n

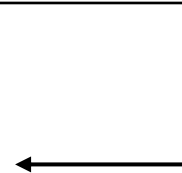


Information Hierarchy



Datum	A single phoneme, a reaction time, or a single token or a sentence.	The smallest, "atomic" unit of observation that cannot be further divided without losing its research meaning.
Data	A collection of reaction times or a list of lemmatized words.	A plural set of observations; the raw material gathered during your computational research.
Dataset	A corpus of ten thousand transcripts or a structured spreadsheet.	A complete, organized collection of data points stored in a single container or flat-file (like a CSV) for analysis.

a single, plain-text table without any structural relationships between different pieces of data. It is **flat** because all the information is squeezed into one dimension, a single table or file



Databases vs Datasets



Feature	Dataset (Flat-File / CSV)	Database (SQLite / Relational)
Memory Usage	RAM-Hungry: Must load the entire file into your computer's "short-term memory."	Lazy-Loading: Only carries the specific "crumbs" of data you ask for at that exact moment.
Integrity	Fragile: A single typo in a CSV can shift entire columns and ruin your "Scientific Depth."	Rigid Schema: Enforces strict rules on data types (e.g., you can't put a word in a "Reaction Time" column).
Speed	Slow Search: Python must read every line from top to bottom to find a specific result.	Indexed Search: Uses a "map" to jump directly to the data, regardless of how large the corpus is.
Stability	High-Anxiety: Likely to crash your script when handling "ten thousand transcripts."	Low-Anxiety: Designed to handle massive, messy real-world data without breaking a sweat.

Python vs SQL



Feature	Python (The Generalist)	SQL (The Specialist)
Language Type	GPL (General Purpose Language)	DSL (Domain-Specific Language)
Control Logic	Imperative & Declarative	Declarative: You define the <i>result</i> (What you want).
Data Handling	Manages logic, loops, and "Scientific Depth" calculations.	Manages the "Geometry of Meaning" through storage and retrieval.
Primary Strength	Flexibility; can build anything from a GUI to an EEG analyzer.	Precision; can filter "ten thousand transcripts" in milliseconds.
Memory State	Lives in the "High-Anxiety" RAM (Short-term memory).	Lives on the "Low-Stress" Disk (Long-term memory).

RDBMS



Database Engine	Architecture Type	Vibe Check	Best For...
Oracle / MS SQL	Enterprise Giant	Very expensive; requires a team of humans in suits to maintain.	Global banking and massive corporate data silos.
MySQL / PostgreSQL	Client-Server	Powerful but "high-anxiety"; needs a dedicated server to "talk" to.	High-traffic websites and multi-user web apps.
	Serverless / Single-File	Low-Stress & Portable: The entire database is just one file on your disk.	Research Pipelines: Sharing corpora and results with the academic community.



Side Quest!

If SQLite is "Serverless" and lives as a single file, what is the biggest advantage for a Master's student submitting a final research project?

Your reward : 1pt

Python Handshake



Method	Syntax	Technical Description
sqlite3	import sqlite3	The built-in Python module required to interface with SQLite databases; no external installation is needed.
.connect()	conn = sqlite3.connect('.db')	Establishes the primary "bridge" to your database file. If the file doesn't exist, SQLite creates it automatically.
.cursor()	cur = conn.cursor()	Generates a "Cursor" object, which acts as the intermediary agent to execute SQL commands and fetch results.
.execute()	cur.execute(" ")	The primary method used by the cursor to send raw SQL strings to the database engine for processing.
.fetchone()	row = cur.fetchone()	A cursor method that retrieves the next <u>single row</u> from a query result, returning it as a Python tuple.
.close()	conn.close()	Terminates the connection to ensure all "Atomic" changes are saved and system memory is released.



```
import sqlite3

# Step 2: Establish the 'Bridge' (Connection) to a database file.
connection = sqlite3.connect('linguistics_study.db')

# Step 3: Hire the 'Librarian' (Cursor).
# The cursor is an object that 'points' to specific data in the colony.
cursor = connection.cursor()

# Step 4: Give the librarian an instruction (Execute).
cursor.execute("SELECT 'Participant_001_Active';")

# Step 5: Retrieve the data (Fetchone).
# .fetchone() grabs the FIRST row of the results and converts it into a Python TUPLE
result = cursor.fetchone()

# Step 6: Use the data in your Python script.
print(f"The Librarian brought back: {result[0]}")

connection.close()
```



Side Quest!

When you use `fetchone()`, the result comes back as a tuple, rather than just a simple string. Why does the Librarian Ant insist on giving you a container instead of just the text?

Your reward : 2pt



SQL Keyword / Type	Syntax Example	Technical Description
CREATE TABLE	CREATE TABLE table_name (...);	Defines the "blueprint" or schema of a new table within the database.
INTEGER	participant_id INTEGER	Stores whole numbers (e.g., age, frequency counts, word length).
TEXT	word TEXT	Stores character strings (e.g., transcripts, lemmas, participant names).
REAL	reaction_time REAL	Stores floating-point numbers (e.g., accuracy percentages, mean RTs).
PRIMARY KEY	id INTEGER PRIMARY KEY	A special constraint that ensures every row has a unique, non-null ID.
IF NOT EXISTS	CREATE TABLE IF NOT EXISTS ...	A safety clause that prevents errors if you run the script multiple times.



```
import sqlite3

# Connect and create the 'bridge'
connection = sqlite3.connect('research_data.db')
cursor = connection.cursor()

# Define the 'Blueprint' for a table of linguistic observations
# We use triple quotes for multi-line SQL strings to keep it clean
table_schema = """
CREATE TABLE IF NOT EXISTS Word_Corpus (
    id INTEGER PRIMARY KEY AUTOINCREMENT, -- Unique ID for every entry
    word TEXT NOT NULL,                  -- The actual word/token
    length INTEGER,                      -- Number of characters
    frequency_score REAL                 -- How common it is in the corpus
);
"""

# Tell the Librarian Ant to build the room
# This creates the structure but doesn't add data yet
cursor.execute(table_schema)
```



SQL Keyword/Method	Syntax	Technical Description
INSERT INTO	INSERT INTO table (cols) VALUES (?);	Adds new observations (rows) to your database chamber.
cursor.execute("SQL COMMAND WITH PLACEHOLDERS", DATA_CONTAINER)		
connection.commit()	conn.commit()	The "Hard Save." Finalizes transactions so they persist on the disk after the script ends.
UPDATE	UPDATE table SET col = val WHERE ...;	Modifies existing "messy real-world data," such as fixing a typo in a participant's response.
DELETE	DELETE FROM table WHERE ...;	Removes specific observations, often used to prune outliers or invalid trial data.



```
# 1. Populating: Adding a new linguistic observation ( We use '?' as placeholders to keep the data secure )
new_word = ("Bilingualism", 12, 0.95)
cursor.execute("INSERT INTO Word_Corpus (word, length, frequency_score) VALUES (?, ?, ?);", new_word)

# 2. THE VITAL STEP: The Hard Save
connection.commit()

# 3. Modifying: Fixing messy real-world data (we realize the frequency score was high)
cursor.execute("UPDATE Word_Corpus SET frequency_score = 0.91 WHERE word = 'Bilingualism';")

# 4. Persistence Check: Final commit for the update
connection.commit()

# 5. Cleaning: Removing an outlier (e.g., a word that was too short)
cursor.execute("DELETE FROM Word_Corpus WHERE length < 2;")
connection.commit()

connection.close()
```

Retrieval and Filtering



Component	Syntax	Technical Description
SELECT	SELECT column1, column2	Identifies the specific variables or "features" you want to extract from the vault.
FROM	FROM table_name	Directs the Librarian Ant to the specific "chamber" where your data lives.
WHERE	WHERE condition	The ultimate filter to "filter the noise" of ten thousand transcripts and isolate specific patterns.
ORDER BY	ORDER BY col DESC/ASC	Sorts your results for structural clarity, ensuring your "Analytic Insight" is organized.
LIMIT	LIMIT 10	Caps the results to prevent system lag when exploring massive, large-scale corpora.

Retrieval and Filtering



```
import sqlite3

# Re-opening our 'digital lab'
conn = sqlite3.connect('research_data.db')
cur = conn.cursor()

# Our goal: Find words that are longer than 5 letters.
# We want to see the most frequent ones first.
query = """
SELECT word, frequency_score
FROM Word_Corpus
WHERE length > 5 AND frequency_score > 0.7
ORDER BY frequency_score DESC
LIMIT 5;
"""
```

```
# The Librarian Ant executes the search
cur.execute(query)

# .fetchall() retrieves ALL rows
# that matched the query as a list of tuples
results = cur.fetchall()

# Displaying our findings
print("--- Top 5 High-Frequency Long Words ---")
for row in results:
    # row[0] is the 'word', row[1] is the 'frequency_score'
    print(f"Word: {row[0]} | Frequency: {row[1]}")

# Always close the vault to keep the lab low-stress
conn.close()
```



Function	Syntax Example	Technical Description
COUNT()	SELECT COUNT(*) FROM trials	Returns the total number of rows (observations) that match your research criteria.
AVG()	SELECT AVG(rt) FROM results	Calculates the arithmetic mean of a numeric column for immediate Analytic Insight .
SUM()	SELECT SUM(tokens) FROM corpus	Computes the total sum of all values in a numeric column.
MIN() / MAX()	SELECT MIN(age), MAX(age) FROM ...	Identifies the lowest and highest values, perfect for defining the range of your Scientific Depth .
AS Alias	SELECT AVG(rt) AS mean_rt ...	Renames the calculated column in your Python output to maintain Code Integrity .

Statistical Depth



```
import sqlite3

# Re-entering the digital laboratory
connection = sqlite3.connect('research_data.db')
cursor = connection.cursor()

# Our Research Goal: Get a statistical summary
# We use 'AS' to make the Python results easy to read
stats_query = """
SELECT
    COUNT(*) AS total_words,
    AVG(frequency_score) AS mean_freq,
    MAX(length) AS longest_word
FROM Word_Corpus;
"""
```

```
# The Ant calculates this on the disk, not in your RAM!
cursor.execute(stats_query)

# we only get one row of results so we use fetchone()
# The result as a tuple: (count, average, max)
stats = cursor.fetchone()

print(f"--- Quick Corpus Stats ---")
# total_words
print(f"Total Observations: {stats[0]}")
# mean_freq
print(f"Average Frequency: {round(stats[1], 2)}")
# longest_word
print(f"Max Character Count: {stats[2]}")
connection.close()
```



SQL Keyword / Concept	Syntax Example	Technical Description
JOIN	FROM TableA JOIN TableB	Combines rows from two or more tables based on a related column between them.
Ambiguity Resolution	Table.Column Name	Using dot notation to tell the "Librarian Ant" exactly which table a shared column name belongs to.
ON	ON TableA.id = TableB.user_id	The "Matchmaker" clause; defines the specific logic used to link the two tables.

Relational Mapping



```
import sqlite3

# Connect to the digital laboratory
conn = sqlite3.connect('bilingual_study.db')
cur = conn.cursor()

# 1. Create two specialized 'chambers' (tables)
# Table A: Demographics
cur.execute("CREATE TABLE IF NOT EXISTS Participants (id INTEGER PRIMARY KEY, name TEXT, l1 TEXT)")
# Table B: Experimental Results
cur.execute("CREATE TABLE IF NOT EXISTS Results (task_id INTEGER, score REAL, p_id INTEGER)")

# 2. Add some 'Atomic' data
cur.execute("INSERT INTO Participants (id, name, l1) VALUES (1, 'Piotr', 'Polish')")
cur.execute("INSERT INTO Results (task_id, score, p_id) VALUES (101, 95.5, 1)")
conn.commit()
```




```
# 3. The Matchmaker: Connecting the tables using JOIN
# We link them ON the Participant ID
query = """
SELECT Participants.name, Participants.l1, Results.score
FROM Participants
JOIN Results ON Participants.id = Results.p_id
"""

cur.execute(query)
data = cur.fetchone() # Fetch the matched pair

# Display the connected insight
# Result: ('Piotr', 'Polish', 95.5)
print(f"Researcher: {data[0]} | Native Lang: {data[1]} | Task Score: {data[2]}")

conn.close()
```